Project 4: Advanced Lane Finding

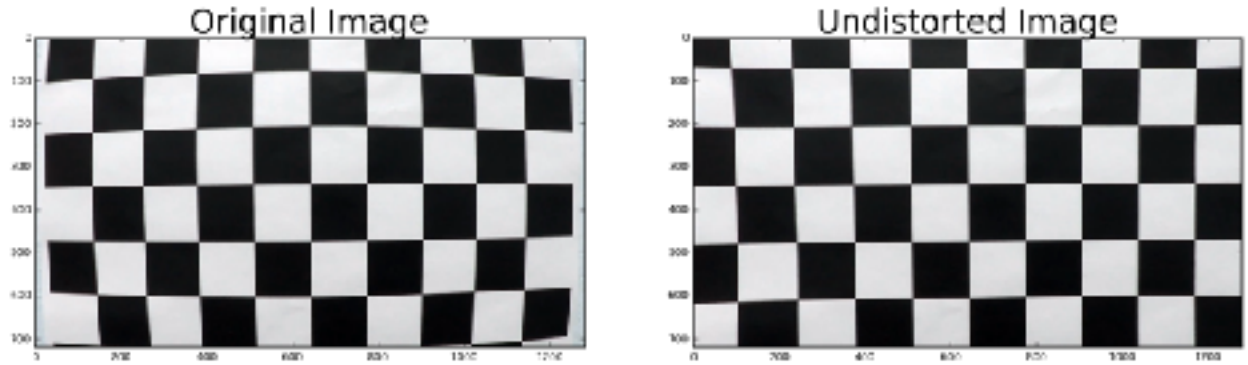
Finding lane lines is an important pipeline to have in self driving cars. In earlier project we are able to find some some form of lines and extrapolate out the lanes we realize that this is not the entirely the best way. Through a series of image transformations and image manipulations we can more accurately get the lane lines along with there curvature. In order for this to work we must build a pipeline that takes in an image and goes through the steps.

The pipeline is below as follows:

- Compute the camera calibration matrix and distortion coefficients given a set of chessboard images.
- Apply a distortion correction to raw images.
- Use color transforms, gradients, etc., to create a thresholded binary image.
- Apply a perspective transform to rectify binary image ("birds-eye view").
- Detect lane pixels and fit to find the lane boundary.
- Determine the curvature of the lane and vehicle position with respect to center.
- Warp the detected lane boundaries back onto the original image.
- Output visual display of the lane boundaries and numerical estimation of lane curvature and vehicle position.

Using lower quality cameras when recording lane lines is a problem because as cars accelerates the frames can have distortion affects and thus can affect the cars ability to see. The best way to go about this it take a series of chessboard images from different angles and create a distortion coefficient matrix. By saving these coefficients we can apply them to other images in our pipeline.

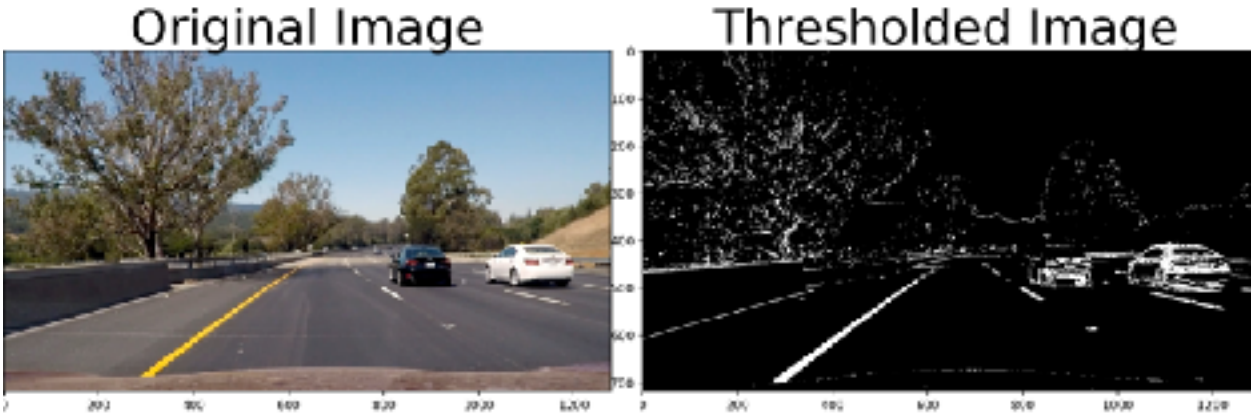
Provide an example of a distortion-corrected image.



The next is to apply this to an actual image and as we can see there are slight modifications to the to the image but indeed there is some adjustment.

Describe how (and identify where in your code) you used color transforms, gradients or other methods to create a thresholded binary image. Provide an example of a binary image result.

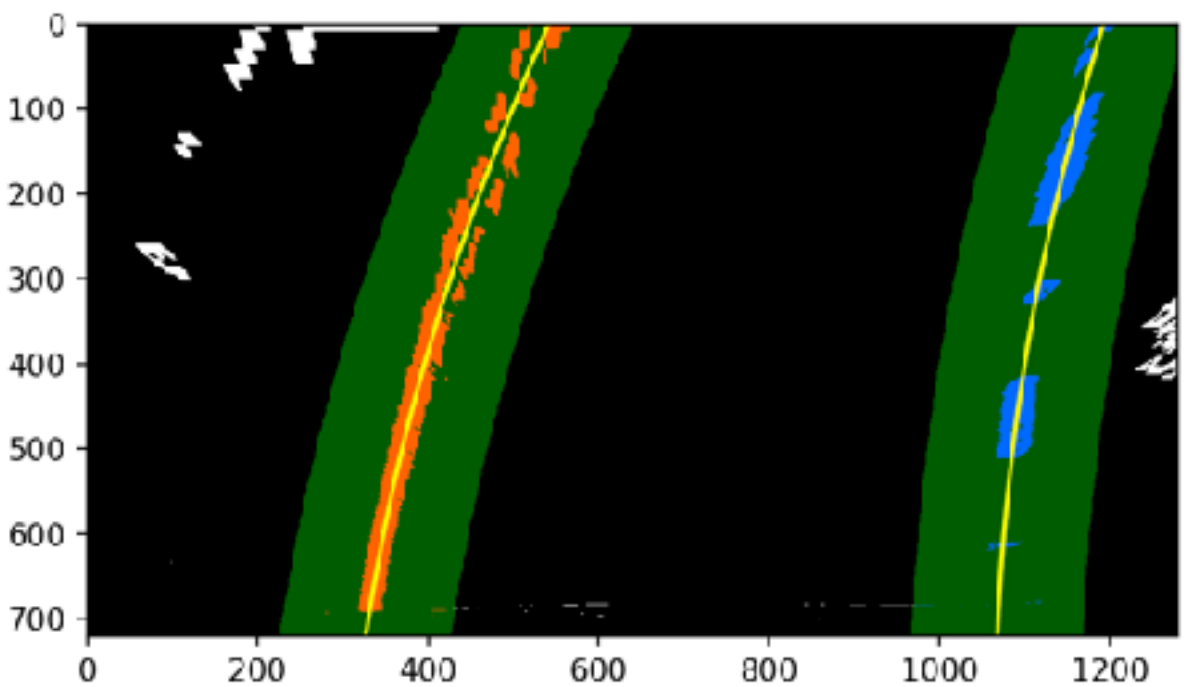
The next step is to apply gradient thresholding. There are two ways of doing this, one is through transformations like sorbet operators that can help find gradient threshold across multiple images and another called color space thresholding. Using these two techniques combined we can show a binarized image is to identify the lane lines. As you can see, the binarized image emphasizes the white lane lines the most. Along with this implementation I used the the R, L, and S



Once we detect the lane lines we must apply a perspective transform. This is seen in the code section where I create a new binary warped image using warp image function.

As we can see, we are able to extract a top down view using the following images as verification. The function returns the source and destination points which can be seen in the console output. The real top-down view will have the gradient thresholding applied to it so that we can more accurately detect the lane lines.

Describe how (and identify where in your code) you identified lane-line pixels and fit their positions with a polynomial?



After receiving the top down view of the image the next step is to apply a polynomial and have it fit to the lane lines. In our code we do this step in the find_lane_lines function. This takes a sliding window approach and over 9 iterations is able to take the mean of the window and create a line.

The last step is to draw the lane lines onto the image. That is done using the function draw lanes on image which takes in the warped image, and the undistorted image and draws the lane lines accordingly.

As can be seen, once the pipeline is implemented it can be used in conjunction with the movie py library. The idea behind the moviepy library is to take a function and apply it to every single frame of a video and recompile that back into a video (A video is just a set of image frames together).

<u>Discussion</u>

A few major problems that were encountered were as follows. When doing camera calibration and distortion we needed to globally save the image points and object points value such that we could access them within the entire scope of the project. This would make it easier for the pipeline to execute and thus not have to apply camera calibration function every single time. Another major issue was finding the right color transformation and sobel thresholds that would work well in each conjunction.

In order to improve my pipeline, I could have used more types of color spaces to get the perfect amount of thresholding. In my current iteration I used directional sobel thresholding and the s channel of HLS color space. My code would perform well on lane markings where trees or shadows are not obstructing the path. In order to adjust for these conditions I could

implement a sort of inference that when creating lane lines look at the previous image and extrapolate. This would help improve my pipeline significantly.